

Slowly Changing Dimension Type 2 (SCD2) – Different Implementations

01

SCD1, SCD2, SCD3

TYPE	DESCRIPTION	PROS	CONS
SCD Type 1	Overwrite old data with new (no history preserved)	Easy to implement	Loses history
SCD Type 2	Adds new row for each change, preserves full history	Tracks complete history	Complex implementation, more storage
SCD Type 3	Adds new row for each change, preserves full history	Limited history with simplicity	Only partial history retained

Different SCD2 Implementations

01 /

Trigger-based

Use database triggers to automatically detect changes in source data and insert a new versioned record into the dimension table. Best for small to medium systems with limited integration points.

02 /

CDC-based

Leverages Change Data Capture (CDC) tools like Oracle GoldenGate to capture changes from the transaction logs and feed them into the data warehouse with full historical tracking.

03 /

ETL-based

Handles change detection and versioning during ETL processes (Extract-Transform-Load), comparing source and target to decide when to insert new history rows.

SCD2

(Slowly Changing Dimension Type 2)





Change Data Capture (CDC) is a technique used to identify and capture changes (INSERTs, UPDATEs, DELETEs) in source databases and deliver them to downstream systems (e.g., data warehouses).

Benefits

- Near real-time tracking
- Minimal impact on source systems
- High performance on large-scale systems

Considerations

- Requires setup
- Monitoring and troubleshooting can be complex
- Might need orchestration layer (e.g., PL/SQL)

SCD2

(Slowly Changing Dimension 2)



ETL

In depth review

Workflow overview of SCD2 & ETL

SCD2 with ETL gives us a powerful, flexible way to capture historical changes in dimension tables — supporting accurate reporting, auditing, and time-based analytics.

The next example of SCD2 & ETL implementation will follow the next steps:

UPDATE

1. Employee updated
2. Detect change
3. Expire current row of the focused record
4. Insert a new version of the record

INSERT

1. New employee added
2. Handle new employees (into dim table)



Why We Use ETL for SCD2 ?

Batch processing:

ETL enable us to compare current data with historical data regularly

Controlled Logic:

We can define PRECISE rules for what counts as CHANGE to be recorded

History preservation:

ETL allows us to insert new rows and expire old ones without losing data

Accurate reporting:

Analysts can see history data at the time of the transaction

Tool-friendly:

ETL tools like ODI, Talend, Informatica have built-in support for SCD2 logic

SCD2

(Slowly Changing Dimension Type 2)





Why you should avoid using SCD type 2 with triggers?

- Triggers introduce **side-effects** that aren't visible in the main data pipeline or ETL jobs.
- Makes **debugging and auditing** difficult because data changes happen behind the scenes.
- Complex trigger logic (like SCD2 logic) is **hard to unit test**, especially with multiple update scenarios.
- Triggers fire **row-by-row**, making them **inefficient** for handling millions of records (typical in data warehouses).
- Poorly written triggers can result in **recursive update chains**, requiring manual intervention or rollback.
- Fragile with changes

Thank *you*!